

Web Application Framework (WAF) - Fall 2026

Task 2 (Deadline: 1st October 2026)

Part 1: Data Generation

1. Generate Employee Data:

- a. Visit www.mockaroo.com and create a dataset containing **at least 7,000 employee records**.
- b. Each record must include the following fields:
 - id (integer, unique, auto-incrementing)
 - fullName (string, e.g., "Umair Bashir")
 - email (string, e.g., "xyz.abc@example.com")
 - department (string, e.g., "Engineering")
 - ipAddress (IPv4 address, e.g., "192.168.1.45")
 - Add a custom field called company (string, e.g., "TechCorp") before generating the data.
- c. Save the generated data as employees.json in your project directory.

2. IP Address Classification:

- Categorize employees based on their ipAddress into the following classes:
 - **Class A:** IP addresses starting with 1-126 (e.g., "10.0.0.1")
 - **Class B:** IP addresses starting with 128-191 (e.g., "172.16.0.1")
 - **Class C:** IP addresses starting with 192-223 (e.g., "192.168.0.1")
 - **Class D:** IP addresses starting with 224-239 (e.g., "224.0.0.1")
 - **Class E:** IP addresses starting with 240-255 (e.g., "240.0.0.1")
- Create the following routes to separate employees by IP class and return their data as JSON:
 - /api/employees/classA
 - /api/employees/classB
 - /api/employees/classC
 - /api/employees/classD
 - /api/employees/classE
- Use the fs module to read from employees.json and filter the data for each route.

Part 2:

Your application must perform the following tasks programmatically (add these as functions in your code):

1. Delete Class E Employees:

- Write a function that removes all employees with Class E IP addresses from employees.json and saves the updated file.

2. Update Class D Employees:

- Write a function that updates the company field to "XYZ Corp" for all employees with Class D IP addresses and saves the updated file.

3. Insert New Employees:

- Generate a separate .json file (e.g., new_employees.json) with **at least 100 new employee records** using Mockaroo (same fields as above). Write a function to append these records to employees.json, ensuring no duplicate id values (reassign ids if necessary).

4. Retrieve Class C Employees:

- Write a function that retrieves all employees with Class C IP addresses and returns them as a JSON array.

Part 3: Web Server and Routes

Create a web server using the Node JS http module. Create four to five routes /, /users, /products, /display, and /books. Maintain a log by appending a file “log.txt” using fs module every time a URL is visited. This file should store information such as serial number, time, date, URL, and number of query parameters for every URL. Create three plain text files named “products.txt”, “users.txt”, and “books.txt” using fs module. Get the information for a specific file from the query parameters and append it. The file “products.txt” should get information from query parameters of route /products and it should contain id, product title, and product price. Similarly the files “users.txt” and “books.txt” should be appended from the query parameters of /users and /books routes, respectively. The file “users.txt” should contain id, user name, age, city, and university. The file “books.txt” should contain id, book title, edition, year of publication, and press name. Hit at least 5 URLs having query parameters such as the following for each of the above-mentioned routes for testing:

/

/users?id=24&name=Abdullah&age=60&city=Islamabad&uni=QAU

/products?id=89&title=Samsung&price=75K

/books?id=19&title=AlgorithmDesignAndApplications&edition=3&2019&press=Wiley

Part 4: NodeJS

1. Use the os module to print information about your computer system's uptime. Also, use the path module to parse and display information about the .js file you are working on.
2. Make sure that you have the following npm packages installed:
 - a. [chalk](#)
 - b. [colors](#)
 - c. [validator](#)
 - d. [prompt](#)You can use `npm list -g --depth=0` command to display the list of installed packages. If any of the packages are not installed, follow the installation instructions from the package's documentation.
3. Write a node script that prompts for your name. Validate against the rule that the first letter of your name should be a capital letter. After the input and validation, display your name on the console with the text color of your choice and the background color of your choice.
4. Write a NodeJS script that:
 - (i) Input an email address and an IPv4 address from the user and validate if the email and the IP are in the correct format.
 - (ii) Generate white- and black list IP addresses and write to corresponding .txt files using fs core module, “White.txt” and “Black.txt”. Your task is to authenticate the user-entered IP against the IP collection given in the files. Use www.mockaroo.com to generate these two files.
 - (iii) If the user-given IP matches an IP in the “Black.txt” list, generate an error communicating that the IP is blocked.
 - (iv) If the user-given IP matches an IP in the “White.txt” list, generate an authentication success message.
 - (v) If the network address of the user-given IP matches the network address of any of the allowed IPs, generate a warning that your IP address belongs to a network whose IP has been authorized. The user should contact the administrator regarding this matter.
 - (vi) If there is none of the above cases, add the user-entered IP into a text

file named "Pending.txt" and display the user an error message communicating that the system is unable to authenticate the IP.

Note: Use an appropriate coloring scheme for displaying messages, i.e., green for success messages, orange for warning messages, and red for error messages. You can use npm documentation for the above mentioned packages.

Part 5: NodeJS Chat

Create a chat room on the local network of the lab.

- i. To send a message, you will use the IP address of the receiver node instead of the local host (192.168.2.1:8080?message="your_message")
- ii. To receive a message, you will create a server on your node listening on port 8080. Use URL module to parse the request url and extract the message.

Using the file system module, keep the history of the chats.

Deliverables:

1. Submit your **Node.js project files**, including:
 - index.js (or equivalent) with all routes and logic.
 - employees.json (initial dataset).
 - new_employees.json (additional dataset).
2. Include a **README.txt** file with:
 - Instructions to run your project (e.g., npm install, node index.js).
3. Ensure your code is well-commented and handles errors (e.g., file not found, invalid JSON).
 - Ensure your application can handle edge cases (e.g., empty files, invalid IP addresses).
4. Submit the compressed folder with the name "yourRollNo_Name_Task2" on fs to the folder WAF-F26-T2. The deadline for this task is 1/10/26 till 16:00.